

Astrology Computation Validation - Technical Sample Brief

This brief summarizes an NDA-safe TypeScript validation sample for a production-bound astrology platform requiring React/TypeScript delivery, backend computational accuracy, Swiss Ephemeris-style validation, and milestone-based acceptance testing.

Technical scope demonstrated

- TypeScript contracts for normalized birth input, chart configuration, planetary positions, house cusps, and aspects.
- Backend-oriented ephemeris adapter boundary so computation remains isolated and the frontend remains a renderer.
- Configurable aspect-grid logic with major/minor aspects and per-aspect orb handling.
- Tolerance-based validation helpers for longitude, latitude, declination, distance, retrograde flags, and house cusps.
- Fixture-based test structure designed to be replaced with Swiss Ephemeris native output and AstroSeek/Astrodienst cross-validation data.

Validation approach

- Primary reference: Swiss Ephemeris native output generated in a controlled environment.
- Cross-validation: AstroSeek and Astrodienst exports for the exact same birth data inputs.
- Regression set: accepted reference cases serialized as JSON fixtures and versioned with the test suite.
- Failure reports should show expected value, actual value, delta, tolerance, source reference, and acceptance status.

Non-determinism audit focus

- Freeze the same local birth input and normalized UTC conversion, then run repeated calculations across backend-only and frontend-triggered paths.
- Compare IANA timezone/DST resolution, ephemeris file path/version, WASM/native behavior, cache/state changes, and Deno cold/warm execution.
- Deliver a root-cause report, bug fix, and a regression test that fails before the fix and passes after the fix.

Architecture preference

- Preserve the existing backend-compute/frontend-render split wherever possible.
- Keep the current WASM wrapper only if it proves deterministic under fixed inputs and fixture-based validation.
- If divergence continues, move Swiss computation behind a controlled native service with a narrow API and comprehensive regression coverage.

Milestone and acceptance discipline

- Each milestone should have executable validation cases or a clear UI acceptance checklist, depending on whether the deliverable is computational or visual.
- For computational features, acceptance should be based on reference fixtures, tolerance thresholds, and repeatable test runs.
- For UI features such as the natal wheel, dashboard, and multi-panel relationship workspace, acceptance should include visual states, responsiveness, accessibility, persistence, and export checks.

NDA-safe note

- This sample does not include proprietary client source code, real birth data, licensed ephemeris files, or commercial secrets.

- It is a compact technical demonstration of the structure and validation discipline that would be applied after NDA, contractor agreement, and codebase access.

Area	Client value
Computation boundary	Keeps risky ephemeris logic testable and isolated from UI rendering.
Reference validation	Makes Swiss Ephemeris, AstroSeek, and Astrodienst comparisons repeatable.
Milestone reporting	Links payment acceptance to objective tests and documented deltas.
UI integration	Supports React/TypeScript chart rendering without mixing visual and computational logic.